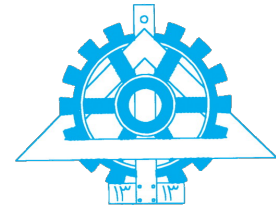




**University of Tehran
College of Engineering
School of Electrical and
Computer Engineering**



Machine Learning

Assignment #4

Student Name
Amirali Dehghani

Student ID
810102443

February 5, 2026

Contents

Abstract	1
1 Question 1: One Full Iteration of K-Means	3
1.1 Assign Data Points to Clusters	3
1.2 Update Cluster Centers	5
1.3 Final Cardinality	5
2 Question 2: Linkage Strategies in Agglomerative Clustering	6
2.1 Computational Complexity	6
2.2 Robustness to Outliers	6
2.3 Clustering with Complete Linkage	7
3 Question 3: Dimensionality Reduction and Data Geometry	10
3.1 Linearly Correlated Gaussian Data	10
3.2 Two Concentric Circles	10
3.3 High-Dimensional Data with Strong Class Overlap . .	11
3.4 Data Containing Many Irrelevant or Noisy Features . .	11
4 Question 4: Formal Theory of LDA and Kernel PCA	13
4.1 LDA Rank Proof	13
4.2 Kernel PCA Derivation	14
5 Question 5: Manual Computation of LDA	17
5.1 Sample Means	17
5.2 Scatter Matrices	17
5.3 Optimal Discriminant Direction	19

5.4	Projected Coordinates	20
6	Question 6: Error Types	21
6.1	Sequential Selection Strategies	21
6.2	Consequences of Greedy Behavior	22
6.3	Comparison with RFE and Lasso	23
7	Question 7: PCA and Kernel PCA on Structured Data	24
7.1	Part (a): PCA on Iris Dataset	24
7.2	Part (b): Nonlinear Data (Nested Circles)	27
8	Question 8: Feature Selection and Visualization on a Real Dataset	29
8.1	Exploratory Data Analysis and Correlation Filtering . .	29
8.2	Univariate Feature Selection (Mutual Information) . .	31
8.3	Recursive Feature Elimination (RFE)	32
8.4	RFE with Cross-Validation (RFECV)	32
8.5	Embedded Method: L1 Regularization (Lasso)	34
8.6	Principal Component Analysis (PCA)	35
8.7	Final Comparative Analysis	36
9	Question 9: Image Segmentation (Bonus)	37
9.1	1K-Means Clustering Results	37
9.2	GMM Clustering Results	38
9.3	Comparative Analysis	38
	References	40

List of Figures

1	Projection of the Iris dataset onto principal components	24
2	Comparison of dimensionality reduction on nested circles	27
3	Distribution of features in the Heart Disease dataset.	29
4	Correlation matrix of the features.	30
5	RFECV performance (Accuracy) versus number of selected features.	33
6	Feature importance ranking based on Lasso coefficients.	34
7	Cumulative explained variance by principal components.	35
8	Segmentation results using K-Means for $K \in \{2, 3, 4, 5\}$.	37
9	Segmentation results using GMM for $K \in \{2, 3, 4, 5\}$. .	38

List of Tables

1	Projected coordinates of data points onto the optimal discriminant direction	20
2	Comparison of Feature Selection Methods	23
3	Classification accuracy comparison (Full vs. MI Selected Features)	31
4	Summary comparison of feature selection methods .	36

Abstract

This assignment explores critical techniques in unsupervised learning, dimensionality reduction, and feature selection, applied to both synthetic and real-world datasets.

First, we investigated the capabilities and limitations of linear versus non-linear dimensionality reduction. On the **Iris dataset**, Standard PCA successfully preserved 95.8% of the variance using two components, though it could not fully separate overlapping classes due to its unsupervised nature. In contrast, on the non-linear **Nested Circles dataset**, Standard PCA failed to unfold the geometry, whereas **Kernel PCA** (using an RBF kernel) successfully mapped the data into a higher-dimensional feature space, achieving linear separability.

Second, we performed a rigorous comparative analysis of feature selection methods on the **Heart Disease dataset**. Exploratory analysis and correlation filtering (threshold > 0.85) revealed no severe multicollinearity. While univariate methods (Mutual Information) achieved an accuracy of 82.4%, **Recursive Feature Elimination (RFE)** outperformed all other methods with an accuracy of **85.4%** by capturing feature interactions. Both RFECV and PCA analyses indicated low redundancy in the data, requiring approximately 12 dimensions to capture 95% of the signal/variance.

Finally, we addressed **Image Segmentation** as a clustering problem using 5-dimensional feature vectors (normalized spatial coordinates and RGB values). We compared **K-Means** and **Gaussian Mixture Models (GMM)**. Results demonstrated that

while K-Means tends to produce geometrically rigid (spherical) segments, GMM offers superior flexibility in capturing complex textures and ellipsoidal clusters, particularly in images with soft boundaries like clouds.

Question 1: One Full Iteration of K-Means

1.1 Assign Data Points to Clusters

We calculate the squared Euclidean distance from each point x_i to the initial centroids $\mu_0 = (5.3, 3.5)$ and $\mu_1 = (5.1, 4.2)$.

- **Point** $x_1 = (5.5, 3.1)$:

$$\begin{aligned}d^2(x_1, \mu_0) &= (5.5 - 5.3)^2 + (3.1 - 3.5)^2 \\&= (0.2)^2 + (-0.4)^2 = 0.04 + 0.16 = 0.20\end{aligned}$$

$$\begin{aligned}d^2(x_1, \mu_1) &= (5.5 - 5.1)^2 + (3.1 - 4.2)^2 \\&= (0.4)^2 + (-1.1)^2 = 0.16 + 1.21 = 1.37\end{aligned}$$

$$0.20 < 1.37 \implies \text{Assign to } C_0$$

- **Point** $x_2 = (5.1, 4.8)$:

$$\begin{aligned}d^2(x_2, \mu_0) &= (5.1 - 5.3)^2 + (4.8 - 3.5)^2 \\&= (-0.2)^2 + (1.3)^2 = 0.04 + 1.69 = 1.73\end{aligned}$$

$$\begin{aligned}d^2(x_2, \mu_1) &= (5.1 - 5.1)^2 + (4.8 - 4.2)^2 \\&= (0)^2 + (0.6)^2 = 0 + 0.36 = 0.36\end{aligned}$$

$$0.36 < 1.73 \implies \text{Assign to } C_1$$

- **Point** $x_3 = (6.6, 3.0)$:

$$\begin{aligned} d^2(x_3, \mu_0) &= (6.6 - 5.3)^2 + (3.0 - 3.5)^2 \\ &= (1.3)^2 + (-0.5)^2 = 1.69 + 0.25 = 1.94 \end{aligned}$$

$$\begin{aligned} d^2(x_3, \mu_1) &= (6.6 - 5.1)^2 + (3.0 - 4.2)^2 \\ &= (1.5)^2 + (-1.2)^2 = 2.25 + 1.44 = 3.69 \end{aligned}$$

$$1.94 < 3.69 \implies \text{Assign to } C_0$$

- **Point** $x_4 = (5.5, 4.6)$:

$$\begin{aligned} d^2(x_4, \mu_0) &= (5.5 - 5.3)^2 + (4.6 - 3.5)^2 \\ &= (0.2)^2 + (1.1)^2 = 0.04 + 1.21 = 1.25 \end{aligned}$$

$$\begin{aligned} d^2(x_4, \mu_1) &= (5.5 - 5.1)^2 + (4.6 - 4.2)^2 \\ &= (0.4)^2 + (0.4)^2 = 0.16 + 0.16 = 0.32 \end{aligned}$$

$$0.32 < 1.25 \implies \text{Assign to } C_1$$

- **Point** $x_5 = (6.8, 3.8)$:

$$\begin{aligned} d^2(x_5, \mu_0) &= (6.8 - 5.3)^2 + (3.8 - 3.5)^2 \\ &= (1.5)^2 + (0.3)^2 = 2.25 + 0.09 = 2.34 \end{aligned}$$

$$\begin{aligned} d^2(x_5, \mu_1) &= (6.8 - 5.1)^2 + (3.8 - 4.2)^2 \\ &= (1.7)^2 + (-0.4)^2 = 2.89 + 0.16 = 3.05 \end{aligned}$$

$$2.34 < 3.05 \implies \text{Assign to } C_0$$

Cluster Assignments:

- $C_0 = \{x_1, x_3, x_5\}$

- $C_1 = \{x_2, x_4\}$

1.2 Update Cluster Centers

New Center for Cluster 0 (μ'_0):

$$\begin{aligned}\mu'_0 &= \frac{x_1 + x_3 + x_5}{3} \\ &= \left(\frac{5.5 + 6.6 + 6.8}{3}, \frac{3.1 + 3.0 + 3.8}{3} \right) \\ &= \left(\frac{18.9}{3}, \frac{9.9}{3} \right) = (6.3, 3.3)\end{aligned}$$

New Center for Cluster 1 (μ'_1):

$$\begin{aligned}\mu'_1 &= \frac{x_2 + x_4}{2} \\ &= \left(\frac{5.1 + 5.5}{2}, \frac{4.8 + 4.6}{2} \right) \\ &= \left(\frac{10.6}{2}, \frac{9.4}{2} \right) = (5.3, 4.7)\end{aligned}$$

1.3 Final Cardinality

The number of points assigned to Cluster 0 is **3**.

Question 2: Linkage Strategies in Agglomerative Clustering

2.1 Computational Complexity

Both Single Linkage and Complete Linkage generally share the same computational complexity. In a standard implementation:

- We calculate the pairwise distance matrix: $O(N^2)$.
- In each iteration, we find the closest clusters and update the matrix.
- The overall complexity is typically $O(N^3)$ for a naive implementation or $O(N^2 \log N)$ with optimized data structures (like heaps).

So, computationally, there is no significant difference between them.

2.2 Robustness to Outliers

Complete Linkage is more robust to outliers.

- **Reason:** Single Linkage uses the minimum distance, which can cause the "chaining effect." A chain of noisy points (outliers) can merge two distinct clusters that are far apart.
- Complete Linkage uses the maximum distance (cluster diameter), forcing clusters to be compact and tightly bound, preventing outliers from easily bridging distant groups.

2.3 Clustering with Complete Linkage

We perform agglomerative clustering using the **maximum distance** between points in clusters.

Step 1: Initial Distance Calculation We calculate Euclidean distances between close points:

$$d(p_4, p_7) = \sqrt{(4-3)^2 + (8-7)^2} = \sqrt{1+1} = \mathbf{1.41}$$

$$d(p_5, p_8) = \sqrt{(8-7)^2 + (3-2)^2} = \sqrt{1+1} = \mathbf{1.41}$$

$$d(p_1, p_2) = \sqrt{(3-1)^2 + (2-1)^2} = \sqrt{4+1} = \mathbf{2.24}$$

$$d(p_3, p_5) = \sqrt{(9-7)^2 + (1-2)^2} = \sqrt{4+1} = \mathbf{2.24}$$

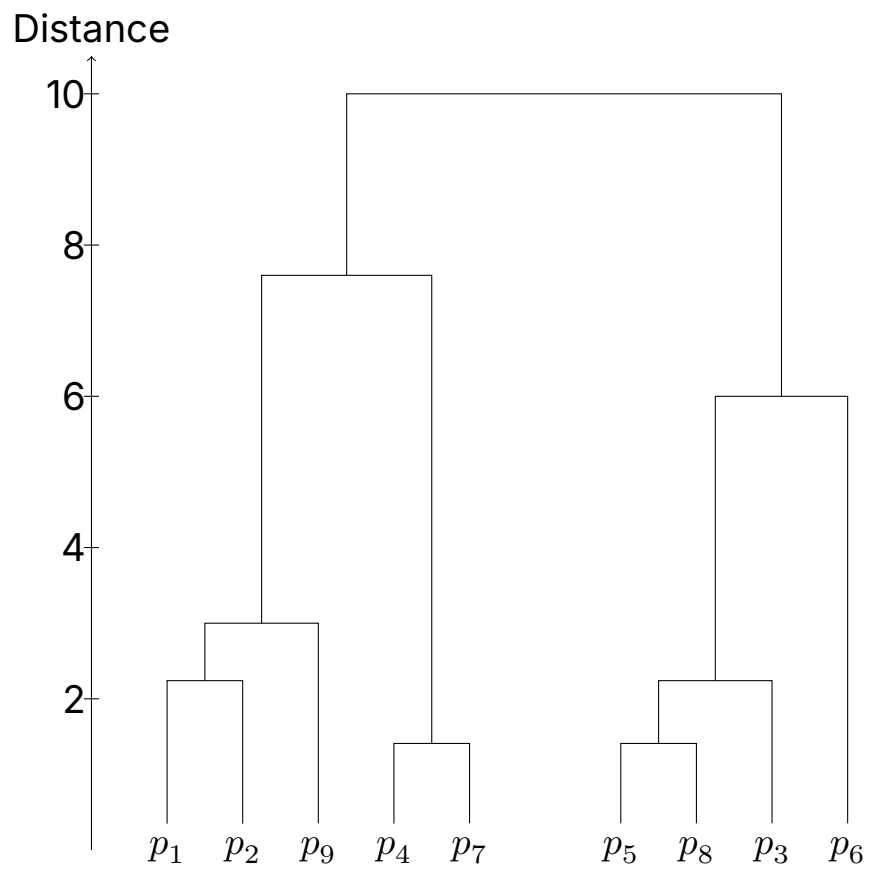
$$d(p_3, p_8) = \sqrt{(9-8)^2 + (1-3)^2} = \sqrt{1+4} = \mathbf{2.24}$$

Step 2: Iterative Merging

- **Merge 1 ($h \approx 1.41$):** Merge $\{p_4, p_7\}$ and $\{p_5, p_8\}$.
- **Merge 2 ($h \approx 2.24$):**
 - Merge $\{p_1, p_2\}$.
 - Merge p_3 with $\{p_5, p_8\}$. (Max dist to p_5 is 2.24, to p_8 is 2.24).
 - Clusters $C_A = \{4, 7\}$, $C_B = \{1, 2\}$, $C_C = \{3, 5, 8\}$, p_6, p_9 . so far:
- **Merge 3 ($h = 3.0$):**
 - Calculate distance from $p_9(1, 4)$ to $C_B\{1, 2\}$.
 - $d(p_9, p_1) = 3$, $d(p_9, p_2) = \sqrt{4+4} \approx 2.8$.
 - Max distance is 3. Merge p_9 into C_B .
 - New Cluster $C_{LeftBottom} = \{1, 2, 9\}$.
- **Merge 4 ($h = 6.0$):**

- Consider $p_6(9, 7)$ and $C_C = \{3, 5, 8\}$.
 - $d(p_6, p_3) = 6$ (Max dist in group).
 - Merge p_6 into C_C .
 - New Cluster $C_{Right} = \{3, 5, 6, 8\}$.
- **Merge 5 ($h \approx 7.6$):**
 - Consider $C_{LeftBottom}\{1, 2, 9\}$ and $C_A\{4, 7\}$.
 - Max dist is between $p_1(1, 1)$ and $p_7(4, 8)$.
 - $d = \sqrt{3^2 + 7^2} = \sqrt{58} \approx 7.6$.
 - New Cluster $C_{Left} = \{1, 2, 9, 4, 7\}$.
- **Final Merge ($h = 10.0$):**
 - Merge C_{Left} and C_{Right} .
 - Max dist between $p_1(1, 1)$ and $p_6(9, 7)$.
 - $d = \sqrt{8^2 + 6^2} = 10$.

Dendrogram:



Question 3: Dimensionality Reduction and Data Geometry

3.1 Linearly Correlated Gaussian Data

Method: PCA (Principal Component Analysis)

- **Geometry:** Gaussian data with linear correlation forms an ellipsoidal shape in space. The principal axes of this ellipsoid correspond to the directions of maximum variance.
- **Reasoning:** PCA is designed to find orthogonal linear projections that maximize variance. Since the data structure is linear and Gaussian, the principal components perfectly capture the underlying structure of the data without needing kernels or class labels.

3.2 Two Concentric Circles

Method: Kernel PCA

- **Geometry:** The data consists of an inner circle and an outer ring. This structure is **not linearly separable**. No straight line or linear plane can separate or unfold these two manifolds effectively.
- **Reasoning:** Standard PCA and LDA are linear methods and would fail to preserve the distinct structure of concentric circles. Kernel PCA uses the "kernel trick" to map data into a

higher-dimensional feature space where these circles become linearly separable (e.g., using a Radial Basis Function kernel).

3.3 High-Dimensional Data with Strong Class Overlap

Method: LDA (Linear Discriminant Analysis)

- **Geometry:** The classes are mixed together. We need a projection that emphasizes the difference *between* classes rather than the overall spread of the data.
- **Reasoning:**
 - **PCA** is unsupervised; it maximizes total variance. In cases of class overlap, the direction of maximum variance might not help separate the classes (it might just stretch the mixed blob).
 - **LDA** is supervised. Its objective function maximizes the *between-class scatter* while minimizing the *within-class scatter*. Even with overlap, LDA attempts to find the specific discriminant direction that best separates the class means.

3.4 Data Containing Many Irrelevant or Noisy Features

Method: PCA (Principal Component Analysis)

- **Geometry:** The informative "signal" usually lies in a lower-dimensional subspace with high variance, while noise is often spread out with lower variance across many dimensions.

- **Reasoning:** PCA separates the signal from the noise by ranking components based on their eigenvalues (variance explained). By keeping only the top principal components and discarding those with small eigenvalues, PCA effectively filters out the random noise and irrelevant features.

Question 4: Formal Theory of LDA and Kernel PCA

4.1 LDA Rank Proof

We define the Between-Class Scatter Matrix S_B as:

$$S_B = \sum_{c=1}^C N_c (\mu_c - \mu)(\mu_c - \mu)^T$$

- C is the number of classes.
- μ_c is the mean vector of class c .
- μ is the global mean vector.
- N_c is the number of samples in class c .

The matrix S_B is a sum of C rank-1 matrices. The vectors involved are $\{(\mu_1 - \mu), (\mu_2 - \mu), \dots, (\mu_C - \mu)\}$. However, these vectors are linearly dependent because the global mean is a weighted average of class means:

$$\mu = \frac{1}{N} \sum_{c=1}^C N_c \mu_c \implies \sum_{c=1}^C N_c (\mu_c - \mu) = 0$$

Since the sum of these vectors is zero, any one vector can be written as a linear combination of the others. Therefore, the set of vectors lies in a subspace of dimension at most $C - 1$.

$$\text{rank}(S_B) \leq C - 1$$

The discriminant directions are the eigenvectors of $S_W^{-1}S_B$ corresponding to nonzero eigenvalues. Since the rank of a product of matrices is limited by the minimum rank of the factors:

$$\text{rank}(S_W^{-1}S_B) \leq \text{rank}(S_B) \leq C - 1$$

Thus, there are at most $C - 1$ nonzero eigenvalues (discriminant directions).

4.2 Kernel PCA Derivation

1. Why direct decomposition is infeasible:

Let $\Phi : \mathbb{R}^d \rightarrow \mathcal{F}$ be the mapping to a high-dimensional feature space. The covariance matrix in $\mathcal{F}$ is:

$$C_{\mathcal{F}} = \frac{1}{N} \sum_{i=1}^N \Phi(x_i) \Phi(x_i)^T$$

If the dimensionality of $\mathcal{F}$ is extremely large (or infinite), computing the size of $C_{\mathcal{F}}$ and its eigen-decomposition is computationally impossible.

2. Principal Component as Linear Combination:

Let v be an eigenvector of $C_{\mathcal{F}}$ with eigenvalue $\lambda > 0$:

$$C_{\mathcal{F}}v = \lambda v$$

Substitute the definition of $C_{\mathcal{F}}$:

$$\left(\frac{1}{N} \sum_{i=1}^N \Phi(x_i) \Phi(x_i)^T \right) v = \lambda v$$

$$\frac{1}{N} \sum_{i=1}^N \Phi(x_i) (\Phi(x_i)^T v) = \lambda v$$

Dividing by λ , we see that v is a linear combination of the mapped samples $\Phi(x_i)$:

$$v = \sum_{i=1}^N \underbrace{\frac{\Phi(x_i)^T v}{N\lambda}}_{\alpha_i} \Phi(x_i) = \sum_{i=1}^N \alpha_i \Phi(x_i)$$

3. Kernel Eigenvalue Problem:

Substitute $v = \sum_{j=1}^N \alpha_j \Phi(x_j)$ back into $\lambda v = C_{\mathcal{F}} v$:

$$\lambda \sum_{j=1}^N \alpha_j \Phi(x_j) = \frac{1}{N} \sum_{i=1}^N \Phi(x_i) \Phi(x_i)^T \left(\sum_{j=1}^N \alpha_j \Phi(x_j) \right)$$

Multiply both sides by $\Phi(x_k)^T$ from the left:

$$\lambda \sum_{j=1}^N \alpha_j \underbrace{\Phi(x_k)^T \Phi(x_j)}_{K_{kj}} = \frac{1}{N} \sum_{i=1}^N \underbrace{\Phi(x_k)^T \Phi(x_i)}_{K_{ki}} \sum_{j=1}^N \alpha_j \underbrace{\Phi(x_i)^T \Phi(x_j)}_{K_{ij}}$$

In matrix notation (K is the kernel matrix):

$$N\lambda K\alpha = K^2\alpha$$

Since K is symmetric and positive semi-definite, we can simplify (assuming K is invertible on the relevant subspace):

$$N\lambda\alpha = K\alpha$$

This is the eigenvalue problem we solve to find α .

4. Projection of a new point:

To project a new data point x onto the principal component v :

$$\text{proj}_v(x) = v^T \Phi(x)$$

Substitute $v = \sum_{i=1}^N \alpha_i \Phi(x_i)$:

$$\text{proj}_v(x) = \left(\sum_{i=1}^N \alpha_i \Phi(x_i) \right)^T \Phi(x) = \sum_{i=1}^N \alpha_i K(x_i, x)$$

Question 5: Manual Computation of LDA

5.1 Sample Means

Class 1:

$$\begin{aligned}\mu_1 &= \frac{1}{5} \sum_{x \in C_1} x = \frac{1}{5} \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 3 \\ 6 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \end{pmatrix} \right] \\ &= \frac{1}{5} \begin{pmatrix} 15 \\ 19 \end{pmatrix} = \begin{pmatrix} 3.0 \\ 3.8 \end{pmatrix}\end{aligned}$$

Class 2:

$$\begin{aligned}\mu_2 &= \frac{1}{5} \sum_{x \in C_2} x = \frac{1}{5} \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} + \begin{pmatrix} 6 \\ 8 \end{pmatrix} + \begin{pmatrix} 9 \\ 5 \end{pmatrix} + \begin{pmatrix} 8 \\ 7 \end{pmatrix} + \begin{pmatrix} 10 \\ 8 \end{pmatrix} \right] \\ &= \frac{1}{5} \begin{pmatrix} 42 \\ 38 \end{pmatrix} = \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}\end{aligned}$$

Overall Mean:

$$\mu = \frac{N_1\mu_1 + N_2\mu_2}{N_1 + N_2} = \frac{5\mu_1 + 5\mu_2}{10} = \frac{\mu_1 + \mu_2}{2} = \begin{pmatrix} 5.7 \\ 5.7 \end{pmatrix}$$

5.2 Scatter Matrices

Within-Class Scatter Matrix (S_W): The within-class scatter matrix is defined as $S_W = S_1 + S_2$.

$$\begin{aligned}
S_1 &= \sum_{x \in C_1} (x - \mu_1)(x - \mu_1)^T \\
&= \begin{pmatrix} 1 & -1.8 \\ -1.8 & 3.24 \end{pmatrix} + \begin{pmatrix} 1 & -0.2 \\ -0.2 & 0.04 \end{pmatrix} + \begin{pmatrix} 1 & 0.8 \\ 0.8 & 0.64 \end{pmatrix} \\
&\quad + \begin{pmatrix} 0 & 0 \\ 0 & 4.84 \end{pmatrix} + \begin{pmatrix} 1 & 0.2 \\ 0.2 & 0.04 \end{pmatrix} \\
&= \begin{pmatrix} 4.0 & -1.0 \\ -1.0 & 8.8 \end{pmatrix}
\end{aligned}$$

$$S_2 = \sum_{x \in C_2} (x - \mu_2)(x - \mu_2)^T = \begin{pmatrix} 9.2 & -0.2 \\ -0.2 & 13.2 \end{pmatrix} \quad (\text{Calculated similarly})$$

$$\begin{aligned}
S_W &= S_1 + S_2 = \begin{pmatrix} 4.0 & -1.0 \\ -1.0 & 8.8 \end{pmatrix} + \begin{pmatrix} 9.2 & -0.2 \\ -0.2 & 13.2 \end{pmatrix} \\
&= \begin{pmatrix} 13.2 & -1.2 \\ -1.2 & 22.0 \end{pmatrix}
\end{aligned}$$

Between-Class Scatter Matrix (S_B):

$$\begin{aligned}
S_B &= N_1(\mu_1 - \mu)(\mu_1 - \mu)^T + N_2(\mu_2 - \mu)(\mu_2 - \mu)^T \\
&= 2.5(\mu_1 - \mu_2)(\mu_1 - \mu_2)^T \quad (\text{Since } N_1 = N_2 = 5)
\end{aligned}$$

$$\text{Diff} = \mu_1 - \mu_2 = \begin{pmatrix} 3.0 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} = \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix}$$

$$\begin{aligned}
S_B &= 2.5 \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix} \begin{pmatrix} -5.4 & -3.8 \end{pmatrix} \\
&= \begin{pmatrix} 72.9 & 51.3 \\ 51.3 & 36.1 \end{pmatrix}
\end{aligned}$$

5.3 Optimal Discriminant Direction

We solve for w that maximizes Fisher's criterion. For binary classification:

$$w \propto S_W^{-1}(\mu_1 - \mu_2)$$

First, compute S_W^{-1} :

$$\det(S_W) = (13.2)(22.0) - (-1.2)(-1.2) = 290.4 - 1.44 = 288.96$$

$$S_W^{-1} = \frac{1}{288.96} \begin{pmatrix} 22.0 & 1.2 \\ 1.2 & 13.2 \end{pmatrix}$$

Now compute w :

$$\begin{aligned} w &\propto \begin{pmatrix} 22.0 & 1.2 \\ 1.2 & 13.2 \end{pmatrix} \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix} \\ &= \begin{pmatrix} (22)(-5.4) + (1.2)(-3.8) \\ (1.2)(-5.4) + (13.2)(-3.8) \end{pmatrix} = \begin{pmatrix} -118.8 - 4.56 \\ -6.48 - 50.16 \end{pmatrix} = \begin{pmatrix} -123.36 \\ -56.64 \end{pmatrix} \end{aligned}$$

Normalized to unit length ($w^* = \frac{w}{\|w\|}$):

$$\|w\| \approx 135.74 \implies w^* \approx \begin{pmatrix} -0.909 \\ -0.417 \end{pmatrix}$$

5.4 Projected Coordinates

Projecting points x onto w^* ($y = w^{*T}x$):

Table 1: Projected coordinates of data points onto the optimal discriminant direction

Class 1 Points	Projection (y)	Class 2 Points	Projection (y)
(4, 2)	−4.47	(9, 10)	−12.35
(2, 4)	−3.49	(6, 8)	−8.79
(2, 3)	−3.07	(9, 5)	−10.27
(3, 6)	−5.23	(8, 7)	−10.19
(4, 4)	−5.30	(10, 8)	−12.43

Question 6: Error Types

6.1 Sequential Selection Strategies

1. Sequential Forward Selection (SFS):

- **Initialization:** Start with an empty set of features $S = \emptyset$.
- **Update Step:** Train the model with each unused feature individually combined with S . Select the feature that improves the performance metric the most and add it to S .
- **Stopping Criterion:** Stop when the desired number of features k is reached or when adding a new feature does not significantly improve performance.

2. Sequential Backward Selection (SBS):

- **Initialization:** Start with the full set of features $S = \{all\ features\}$.
- **Update Step:** Train the model by temporarily removing each feature from S . Permanently remove the feature whose absence causes the least drop (or improvement) in performance.
- **Stopping Criterion:** Stop when k features remain or when removing a feature significantly hurts performance.

3. Bidirectional (Stepwise) Selection:

- This method combines SFS and SBS. It starts like SFS, adding the best feature. However, after each addition, it checks if any of the previously selected features can be removed without degrading performance (like SBS). This allows the algorithm to correct early mistakes.

Why "Greedy"?

These algorithms are considered **greedy** because at each step, they make the locally optimal decision (adding the best current feature or removing the worst current one) with the hope that these local choices lead to a global optimum. They do not backtrack or evaluate all possible subsets (which would be exhaustive search).

6.2 Consequences of Greedy Behavior

Greedy methods like SFS and SBS are **not guaranteed** to find the globally optimal feature subset. This is because they ignore feature interactions (dependencies) that only appear when specific features are present together.

Example Scenario (The XOR Problem):

Consider a dataset where the target Y is determined by the XOR function of two features x_1 and x_2 (i.e., $Y = x_1 \oplus x_2$).

- Individually, x_1 has 0 correlation with Y (random guessing).
- Individually, x_2 has 0 correlation with Y .
- Together, they perfectly predict Y .

Failure of SFS: In the first step, SFS evaluates x_1 and x_2 individually. Since neither provides predictive power on its own, SFS might discard both and select a noisy feature x_3 that has a

slight accidental correlation. The algorithm never sees the optimal pair $\{x_1, x_2\}$ because it made a greedy decision based on single-feature performance.

6.3 Comparison with RFE and Lasso

Table 2: Comparison of Feature Selection Methods

Criteria	Sequential (SFS/SBS)	RFE (Recursive Feature Elimination)	L1 Regularization (Lasso)
Complexity	High. Requires training the model many times ($O(d^2)$ for SFS).	High to Medium. Trains the model iteratively, removing features step-by-step.	Low. Solves a single optimization problem (convex). Very efficient.
Correlations	Can be unstable. Might pick one correlated feature and ignore the other arbitrarily.	Handles correlations reasonably well but can be computationally heavy.	Handles correlations well by arbitrarily selecting one and shrinking the coefficient of the other to zero.
Interactions	Poor. Misses interactions if individual features are weak (e.g., XOR).	Better. Since it starts with all features, it can preserve interacting groups before removing them.	Limited. Linear models (standard Lasso) cannot capture non-linear interactions unless explicitly engineered.
High Dim.	Not suitable. Too slow if number of features is very large.	Can be used, but training the full model initially can be slow.	Highly suitable. Can handle cases where features $>$ samples effectively.

Question 7: PCA and Kernel PCA on Structured Data

7.1 Part (a): PCA on Iris Dataset

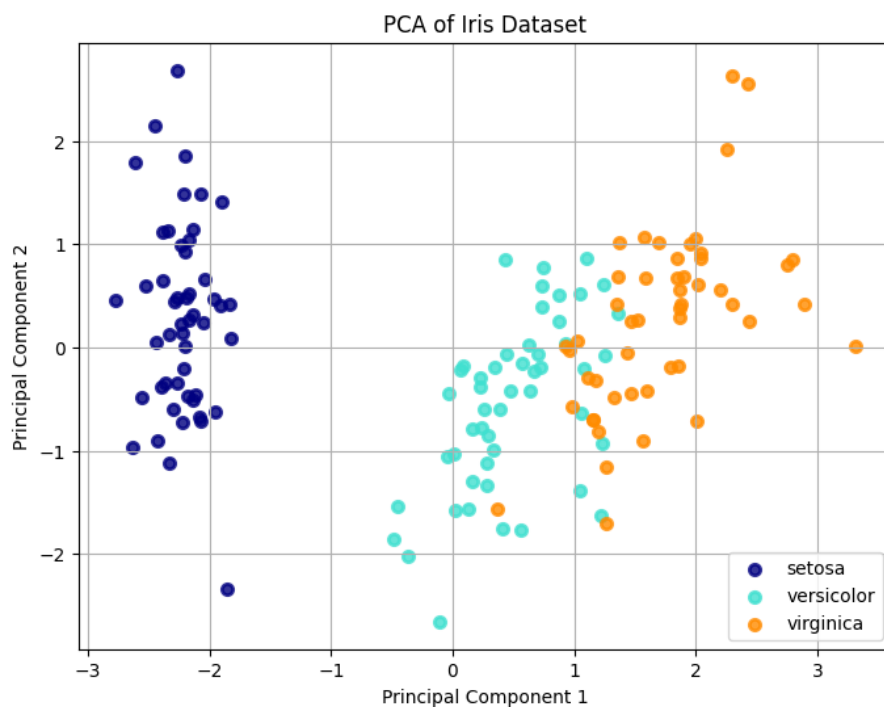


Figure 1: Projection of the Iris dataset onto the first two principal components. The colors represent the three different classes: Setosa, Versicolor, and Virginica.

1. Eigenvalues and Explained Variance

Based on the computational results, the eigenvalues of the covariance matrix and the explained variance ratios are:

- **Eigenvalues:** $\lambda_1 \approx 2.94$, $\lambda_2 \approx 0.92$

- **Explained Variance Ratio:**

PC1: 72.96%, PC2: 22.85%

- **Total Variance Explained:** $\approx 95.81\%$

This indicates that reducing the data to 2 dimensions preserves the vast majority of the information.

2. Analysis of Principal Components

The first principal component (PC1) captures the largest amount of variance (72.96%). Intuitively, this component aligns with the direction in the 4D feature space where the data points are most spread out. In the context of the Iris dataset, this is largely driven by the petal length and petal width features, which are highly correlated and vary significantly between classes.

3. Class Separability and PCA Objective

Observing the 2D plot:

- The **Setosa** class (dark blue) is well-separated from the others.
- The **Versicolor** and **Virginica** classes show significant overlap in the middle.

This outcome is expected because PCA is an **unsupervised** method. Its objective function maximizes the **total variance** of the projection ($\text{Var}(Xw)$), without any knowledge of class labels. It does not attempt to minimize intra-class variance or maximize the distance between class means. The separation of Setosa is incidental due to its naturally distinct feature values, while the overlap of the other two indicates that the direction of maximum variance is not necessarily the direction of best discrimination.

4. Suitability for Classification

PCA is not always the most appropriate technique if the sole goal is classification. As seen here, it preserves the "spread" but not necessarily the "separability." If the discriminant information lies in a direction of low variance (e.g., orthogonal to the main spread), PCA might discard it, leading to poor classification performance in the reduced space.

5. Conceptual Comparison with LDA

If Linear Discriminant Analysis (LDA) were applied instead:

- **Objective Function:** Unlike PCA, LDA is **supervised**. It explicitly attempts to maximize the Between-Class Scatter (S_B) while minimizing the Within-Class Scatter (S_W).
- **Projection Effect:** LDA would identify a projection axis that pushes the means of the three classes as far apart as possible while making each cluster as compact as possible.
- **Result:** We would conceptually expect the overlap between Versicolor and Virginica to be significantly reduced or eliminated in the LDA space, as the algorithm would specifically target the features that distinguish these two classes, rather than just the features that vary the most.

7.2 Part (b): Nonlinear Data (Nested Circles)

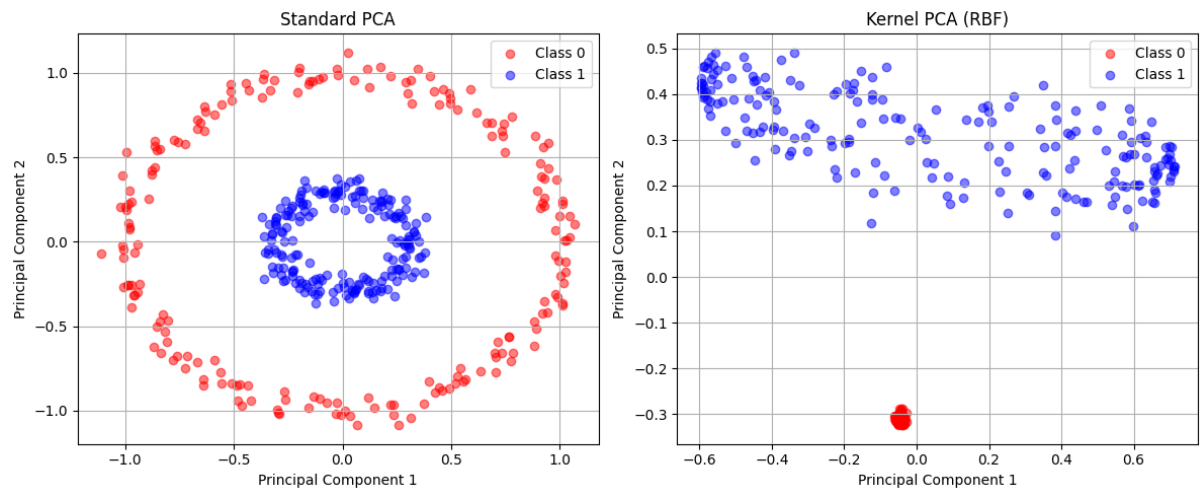


Figure 2: Comparison of dimensionality reduction on nested circles. Left: Standard PCA (Linear) keeps the concentric structure, failing to separate classes. Right: Kernel PCA (RBF) unfolds the data into distinct clusters, achieving linear separability.

1. Comparison of Results

The visualization compares the performance of linear and non-linear techniques:

- **Standard PCA (Left Plot):** The projection preserves the original concentric geometry. The red class (inner circle) is still surrounded by the blue class (outer ring). Since PCA is restricted to linear transformations (rotation/scaling), it fails to unfold this non-linear structure, leaving the classes mixed and not linearly separable.
- **Kernel PCA (Right Plot):** The method successfully untangles the data. As seen in the right panel, the red and blue points are mapped to distinct regions in the feature space. Unlike the left plot, the two classes are now clearly separated and could be distinguished by a simple linear classifier.

2. The Role of Nonlinear Mapping and the Kernel Trick

Standard PCA fails because it assumes the data lies on a linear subspace, whereas the concentric circles form a non-linear manifold. Kernel PCA overcomes this limitation by implicitly mapping the input data $x \in \mathbb{R}^2$ into a high-dimensional feature space $\mathcal{F}$ using a non-linear mapping $\Phi(x)$.

By using the **kernel trick** (specifically the RBF kernel), we compute the principal components in this high-dimensional space without explicitly calculating the coordinates. The kernel function measures the similarity between points in the transformed space, effectively "lifting" or "unfolding" the classes so they become linearly separable, as demonstrated in the RBF projection.

Question 8: Feature Selection and Visualization on a Real Dataset

8.1 Exploratory Data Analysis and Correlation Filtering

Feature Distributions:

The dataset contains 1025 samples and 14 features. The histograms in Figure 3 illustrate the distribution of each feature. We observe a mix of continuous variables (e.g., age, cholesterol, thalach) and categorical variables encoded as integers (e.g., sex, cp, slope). The target variable appears to be relatively balanced.

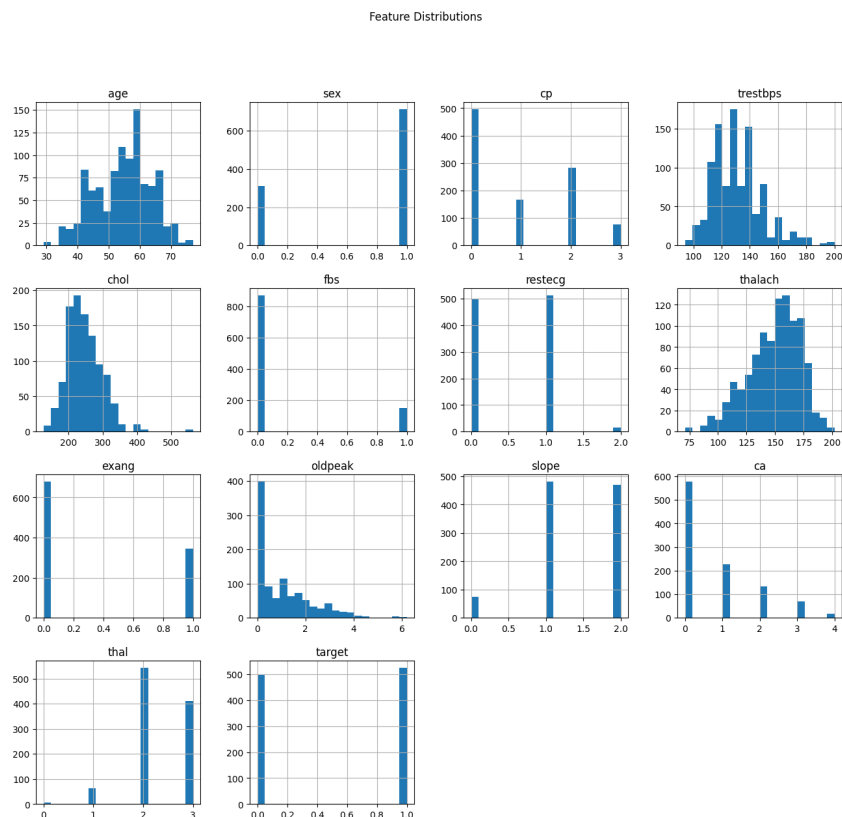


Figure 3: Distribution of features in the Heart Disease dataset.

Correlation Analysis:

The correlation matrix (Figure 4) reveals the linear relationships between features.

- The target variable shows moderate positive correlation with `cp` (chest pain type, 0.43) and `thalach` (max heart rate, 0.42).
- It shows negative correlation with `exang` (exercise induced angina, -0.44) and `oldpeak` (-0.44).
- Among the input features, `oldpeak` and `slope` have a notable negative correlation (-0.58), but no pair exhibits extreme multicollinearity.

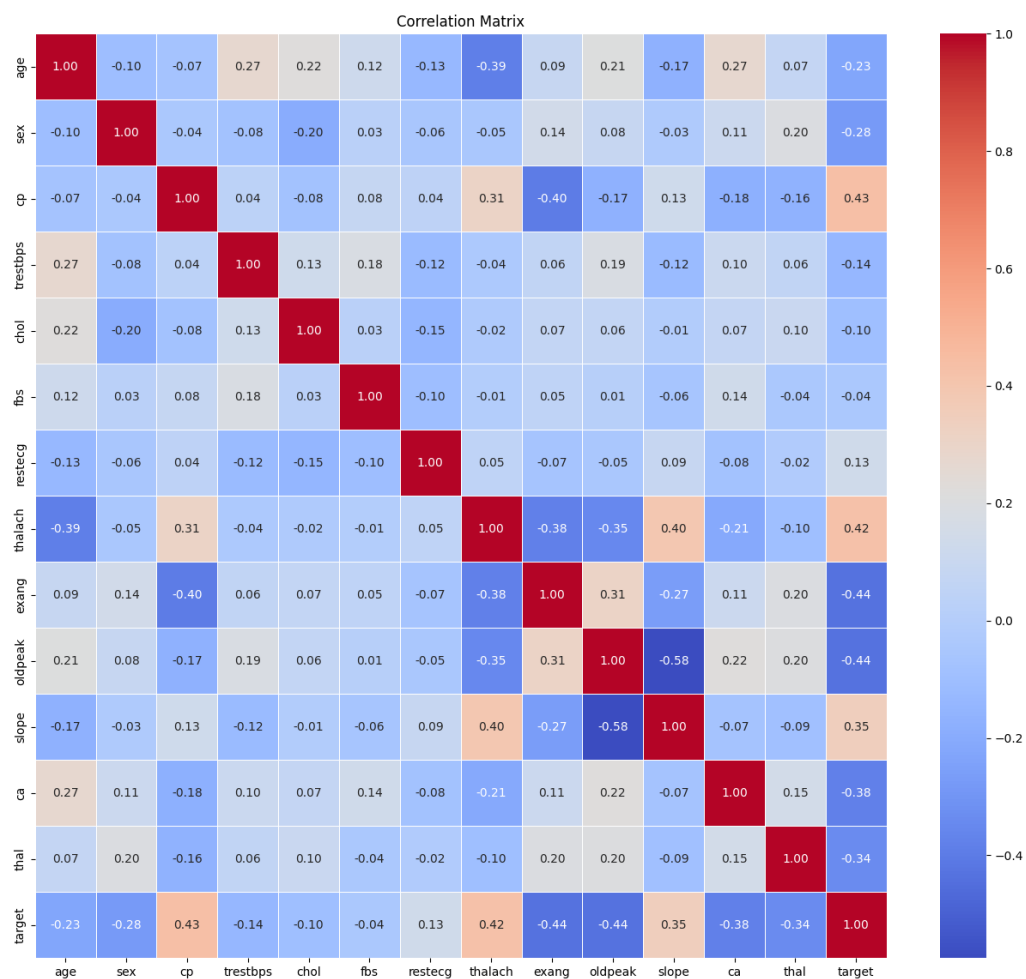


Figure 4: Correlation matrix of the features.

Outcome of Correlation-based Filtering:

We applied a correlation threshold of 0.85 to identify redundant features.

Features to remove: [] (None)

Since no pair of features had a correlation coefficient exceeding the threshold, **no features were removed**. This suggests that while some features are related, the dataset does not suffer from severe redundancy or perfect collinearity that would necessitate removing features purely based on pairwise correlation.

8.2 Univariate Feature Selection (Mutual Information)

We applied Mutual Information (MI) to rank the features based on their individual dependency with the target variable. The top $k = 5$ features selected are:

`[cp, chol, thalach, oldpeak, thal]`

Performance Comparison: We trained a Logistic Regression model on the full feature set versus the subset selected by MI.

Table 3: Classification accuracy comparison (Full vs. MI Selected Features)

Feature Set	Accuracy
All Features (13 features)	0.8098
Top-5 MI Features	0.8244

Analysis: Interestingly, the model trained on the reduced feature set outperformed the model trained on all features (82.44% vs 80.98%). This suggests that some input features act as noise or are irrelevant, and their removal helps the model generalize better.

8.3 Recursive Feature Elimination (RFE)

Next, we applied RFE using Logistic Regression as the estimator to select the top 5 features. Unlike univariate methods, RFE considers feature interactions implicitly by pruning the least important feature from the model iteratively.

Selected Features by RFE:

[sex, cp, exang, oldpeak, ca]

Comparison with Univariate Method:

- **Accuracy:** The RFE-selected subset achieved an accuracy of **0.8537**, which is significantly higher than both the full model and the MI subset. This demonstrates the superiority of multivariate selection strategies that account for feature dependencies.
- **Feature Overlap:** The intersection between MI and RFE is:

Overlap = {cp, oldpeak}

Only 2 out of 5 features are common. MI favors features with high individual correlation (like `chol` and `thalach`), whereas RFE selects features that, *in combination* (like `sex` and `ca`), provide the best decision boundary.

8.4 RFE with Cross-Validation (RFECV)

To find the optimal number of features automatically and robustly, we used RFECV with Stratified K-Fold cross-validation.

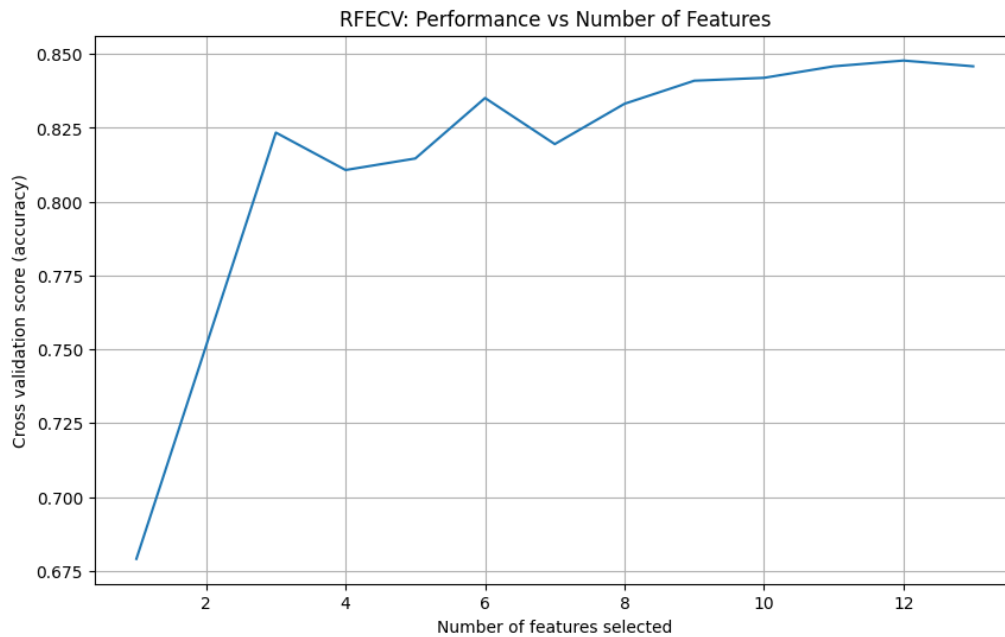


Figure 5: RFECV performance (Accuracy) versus number of selected features.

Results:

- **Optimal Number of Features:** 12
- **Selected Features:** Almost all features were selected, except for `fbs` (Fasting Blood Sugar).
- **Interpretation:** The cross-validation score peaks at 12 features. The fact that RFECV retained nearly all features implies that most attributes in the Heart Disease dataset contribute unique and useful information for classification. The removed feature, `fbs`, likely has very weak predictive power (as hinted by its near-zero correlation in the heatmap).

8.5 Embedded Method: L1 Regularization (Lasso)

We applied L1-regularized Logistic Regression (Lasso) to select features and evaluate their importance based on the magnitude of their coefficients.

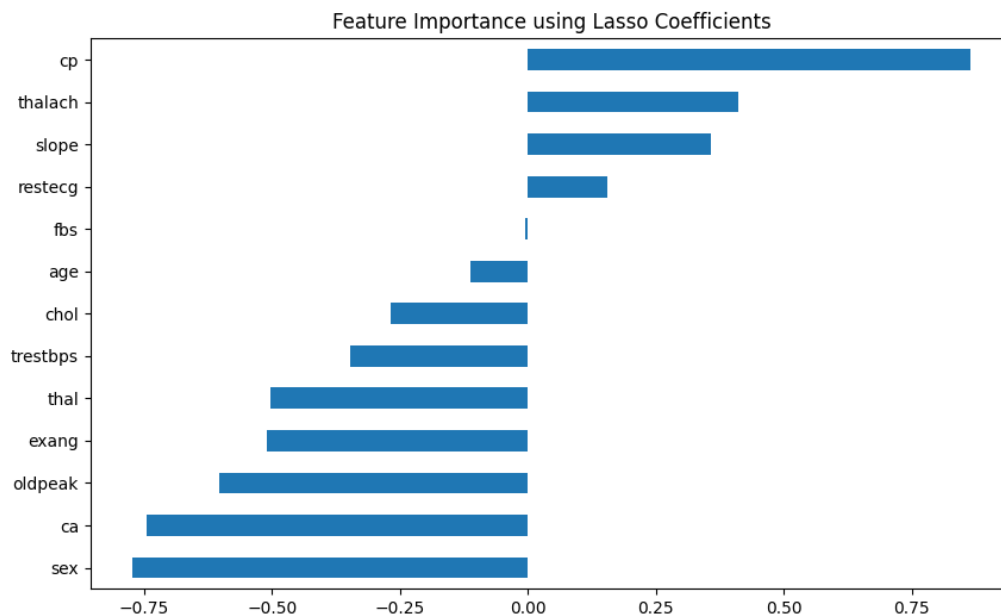


Figure 6: Feature importance ranking based on Lasso coefficients.

Results:

- **Selected Features:** All 13 features were retained (Sparsity = 0).
- **Ranking:** The features with the largest absolute coefficients are `cp` (Chest Pain), `sex`, and `ca`.
- **Comparison with RFE:** There is a strong consensus between Lasso and RFE. The top features identified by Lasso (highest bars) are exactly the ones selected by RFE ('cp', 'sex', 'ca', 'oldpeak', 'exang'). This confirms that these features are the most robust predictors of heart disease in this dataset.

8.6 Principal Component Analysis (PCA)

As an exploratory tool, we applied PCA to determine the intrinsic dimensionality of the dataset.

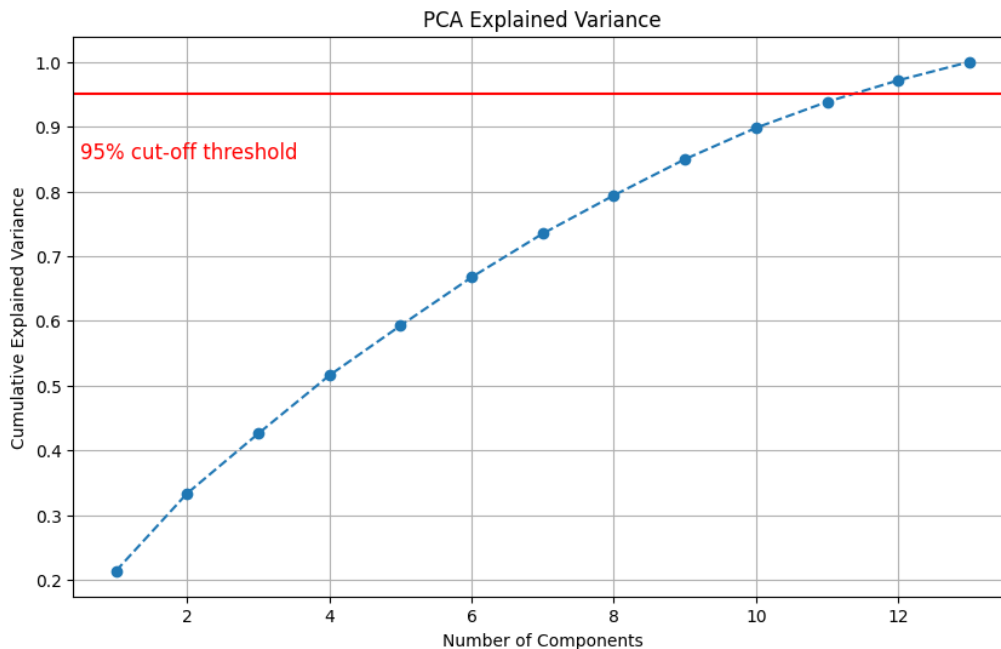


Figure 7: Cumulative explained variance by principal components.

Dimensionality Analysis:

- **95% Variance Threshold:** It requires **12 components** to explain 95% of the variance.
- **Interpretation:** Since we need 12 out of 13 possible dimensions to capture the signal, this indicates that the features are not highly linearly redundant. This explains why the correlation filter (Section 1) failed to remove any features and why RFECV (Section 4) kept almost all features. The "information content" is spread across nearly all collected attributes.

8.7 Final Comparative Analysis

Table 4: Summary comparison of feature selection methods

Method	Outcome	Key Observation
Correlation Filter	No features removed	Ineffective for this dataset due to lack of strong pairwise linear correlations (> 0.85).
Univariate (MI)	Top-5 Accuracy: 82.4%	Simple and fast, but ignores feature interactions. Missed meaningful features like 'sex'.
RFE (Wrapper)	Top-5 Accuracy: 85.4%	Best Performance. Successfully captured interacting features (e.g., 'sex' + 'ca') that univariate methods missed.
RFECV	12 Features Selected	Confirmed that most features contain useful signals, preventing aggressive dimensionality reduction.
Lasso (L1)	All Features Retained	Provided a feature ranking consistent with RFE but failed to induce sparsity with the default regularization parameter.
PCA	12 Components for 95%	Confirmed low redundancy. Unlike feature selection, PCA transforms features, losing physical interpretability (e.g., we lose the meaning of "Age").

Conclusion: For this specific dataset, **RFE** proved to be the most effective method, yielding the highest classification accuracy with a compact subset of 5 features. While PCA and RFECV suggest that information is spread across all dimensions, RFE demonstrated that a small, carefully selected subset of interacting features is sufficient for building a high-performance predictive model.

Question 9: Image Segmentation (Bonus)

In this task, we performed image segmentation using two unsupervised learning algorithms: K-Means and Gaussian Mixture Models (GMM). For both methods, each pixel was represented as a 5-dimensional feature vector:

$$x_i = [y_{norm}, x_{norm}, R_{norm}, G_{norm}, B_{norm}]$$

This creates a feature space that accounts for both the spatial locality and the color similarity of pixels.

9.1 1K-Means Clustering Results

Figure 8 displays the segmentation results using the K-Means algorithm with Euclidean distance assignments.

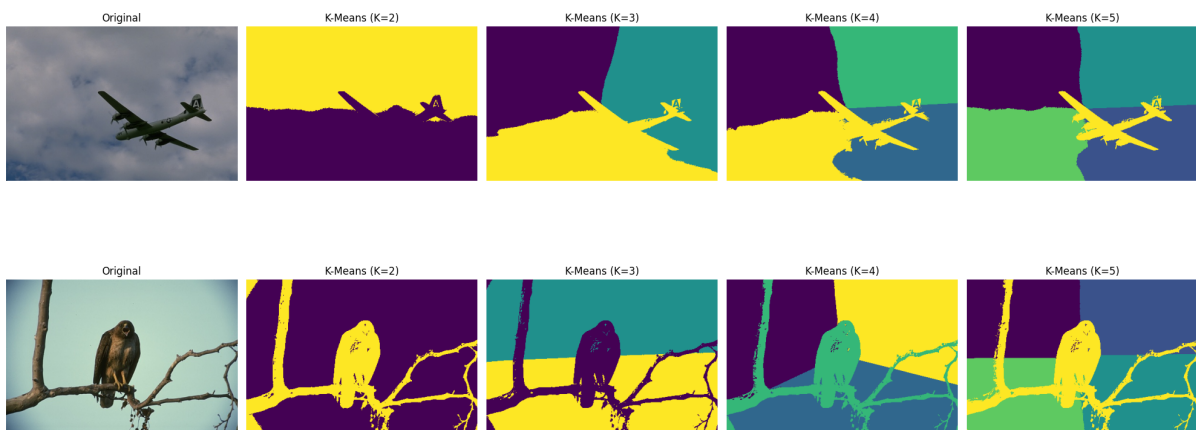


Figure 8: Segmentation results using K-Means for $K \in \{2, 3, 4, 5\}$.

9.2 GMM Clustering Results

Figure 9 shows the segmentation results using Gaussian Mixture Models, trained via the Expectation-Maximization (EM) algorithm.

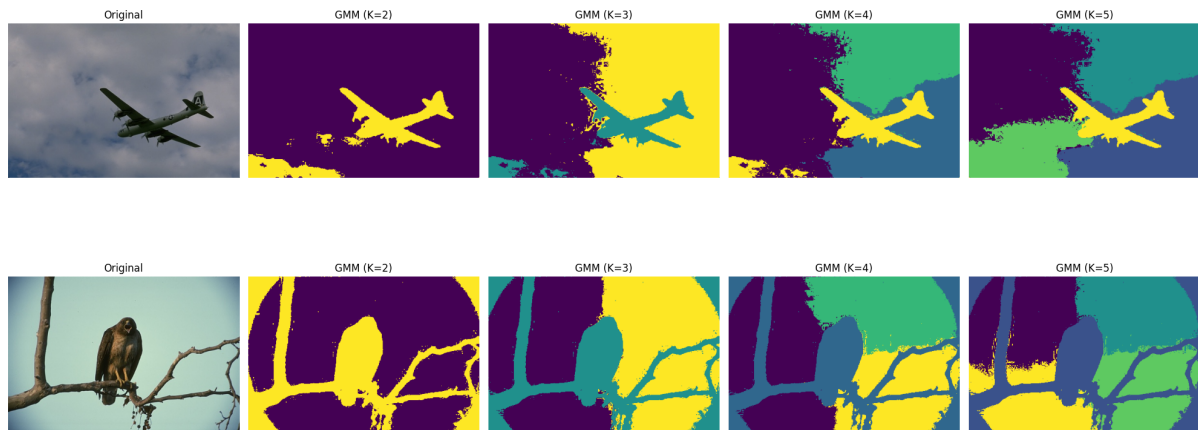


Figure 9: Segmentation results using GMM for $K \in \{2, 3, 4, 5\}$.

9.3 Comparative Analysis

Comparing the two methods reveals distinct behaviors due to their underlying assumptions:

- **Cluster Geometry:** K-Means uses Euclidean distance, which implicitly assumes that clusters are spherical and have equal variance. This tends to produce "hard," geometric boundaries. In the plane image (top row of Figure 8), the sky is segmented into large, smooth blocks.
- **Flexibility and Texture:** GMM assumes the data is generated from a mixture of Gaussian distributions. By learning the full covariance matrix, GMM allows for ellipsoidal clusters of varying sizes and orientations. This makes GMM more sensitive to color variances.

- **Visual Observation:** In the GMM results (Figure 9), the segmentation of the sky is "noisier" or more granular compared to K-Means. It picks up the subtle texture of the clouds (variance in white/blue) rather than smoothing them out into a single block. While K-Means favors spatial coherence (due to the x, y features dominating in a spherical metric), GMM's flexibility allows it to adapt better to the complex distribution of color intensity, resulting in more detailed, albeit sometimes speckled, segments.

References

1. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Reference for Principal Component Analysis, Gaussian Mixture Models, and the EM algorithm).
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. (Reference for Feature Selection methods, Recursive Feature Elimination, and Lasso regularization).
3. Schölkopf, B., & Smola, A. J. (2002). *Learning with Kernels*. MIT Press. (Reference for Kernel Principal Component Analysis and non-linear feature mapping).
4. Dua, D. & Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences. (Source for the Iris and Heart Disease datasets used in the experiments).
5. Scikit-Learn Documentation. Available at: <https://scikit-learn.org/stable/>. (Used for implementation of PCA, RFE, RFECV, and Clustering algorithms).